FA HACKATHON · TRACK: NON-AGENT (AUTOMATION & ANALYTICS)

Treasury Payment Control Tower

From manual payment checking to automated treasury control.

7	6	3	1
legal entities	currencies	banks	screen

Individual entry · Lê Thị Cẩm Tú (Stella) — Oversea Treasury

Why Non-Agent: deterministic parsing, rule-based reconciliation and control logic — correctness and auditability matter more than generative output. Live demo: tultc4.github.io/treasury-payment-dashboard

PAIN POINT

One payment run, five disconnected files, zero visibility

The risk isn't effort — it's that errors and funding gaps are found late, by hand, or not at all.

AP Excel (× 7 entities) – **Manual line** check – **Manual** enrichment – **ERP file** by hand – **Compare bank** file by eye – **Search statement** per debit – **Balance check** per account

WHAT CAN GO WRONG — SILENTLY

- Wrong or stale beneficiary account / SWIFT
- Duplicate payment reference vs. bank history
- Wrong value date · holiday or cut-off missed
- Funding shortfall discovered after submission
- A “settled” payment that never actually debited

WHO PAYS FOR IT

Treasury — a full day per cycle re-checking

CFO — payment & liquidity risk

Auditor — no single evidence trail

Vendors — late payments

SOLUTION

One control tower over the whole payment lifecycle

Files go in, controlled decisions come out — a human stays at every control point.

INPUT

- AP payment lists
- ERP upload template
- Bank creation file
- Bank statements
- Master data & balances
- Holiday / run calendar

→

PROCESSING — AUTOMATED

- Parse & validate every row
- Enrich: bank · method · charges
- Reconcile bank file (4 fields)
- Match debits transaction-level
- Cash sufficiency per account
- Exception detection · audit logging

→

OUTPUT

- Clean ERP CSV per entity
- Live status: AP → debit
- Exception list (owner · age)
- Funding-gap alert
- Approval email + evidence
- Audit trail of every action

HUMAN CONTROL POINTS — NOTHING PAYS AUTOMATICALLY

Treasury reviews enrichment → approves ERP export → resolves every bank DIFF (override needs a written reason) → approval email to L1/L2 before release

IMPACT

From a day of checking to minutes of deciding

Measurable time back — and risks that used to be invisible are caught before money moves.

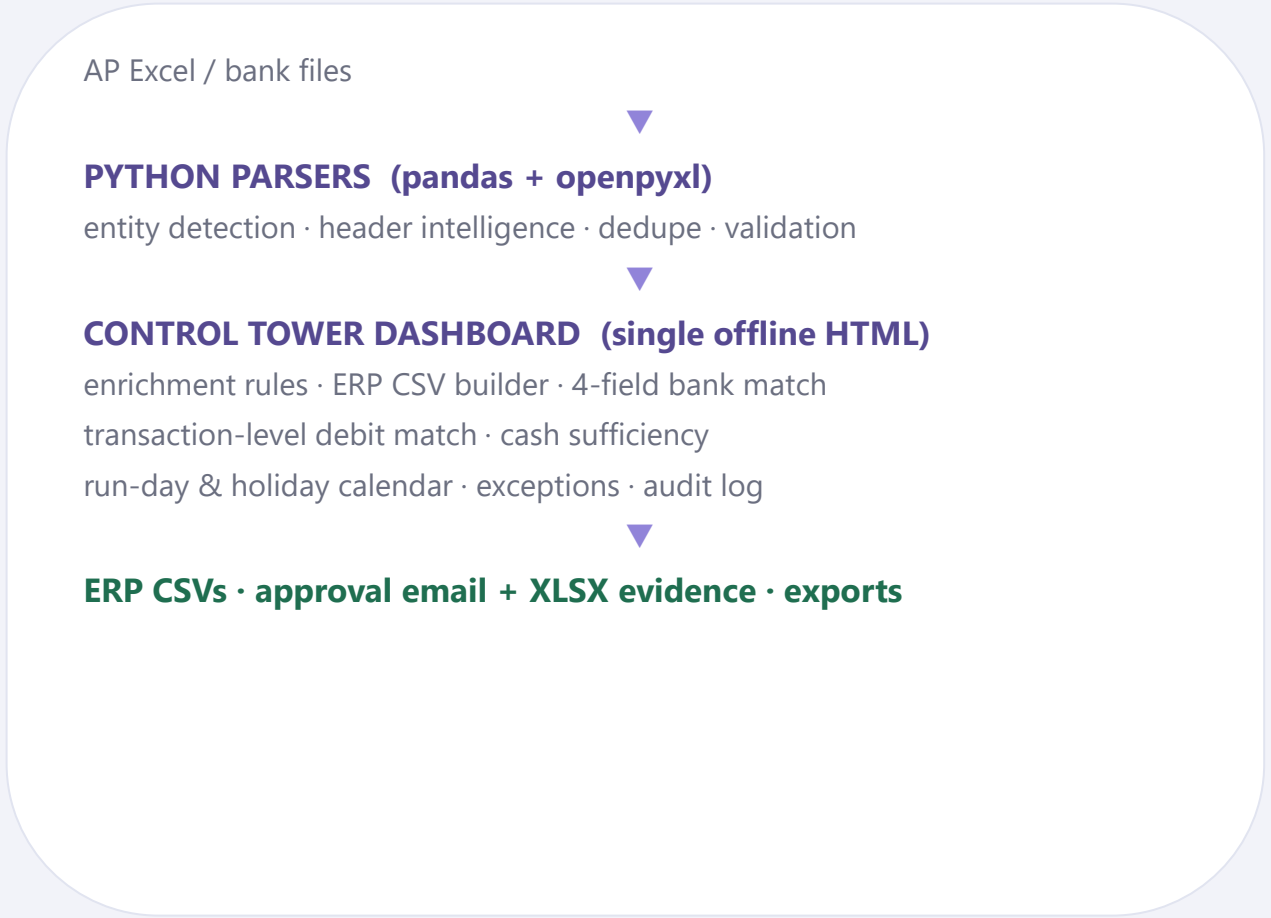
BEFORE — MANUAL	AFTER — CONTROL TOWER	KPI TO REPORT
Line-by-line checking, 7 entities	46 groups parsed & validated in under a minute	**XX hrs → XX min / cycle**
Bank file compared by eye	4-field auto-match; only true DIFFs surface	**XX% fewer manual checks**
Debits hunted in statement PDFs	333 statement lines matched automatically	**XX% faster settlement**
Funding gaps found after submission	Shortfall alert before the run (real ~5M THB catch)	**100% pre-run coverage**
Errors found downstream — or never	Duplicates, missing fields, zero amounts blocked at source	**XX reworks avoided / qtr**
Audit evidence reconstructed by hand	Every import, edit, override, approval logged (who + when)	**100% action traceability**

How we measure: stopwatch two cycles side-by-side (manual vs. dashboard, same batch, same operator) · count manual touchpoints per payment · timestamp issue-in-file → issue-surfaced · reworks per quarter from ERP & bank logs.

ARCHITECTURE

Two parsers, one file — zero data leaves Treasury

The architecture is a control decision — privacy by design, near-zero cost, honest about production gaps.



AUTOMATED

parsing, validation, enrichment defaults, matching, sufficiency, calendar, logging

HUMAN APPROVAL

ERP export · every DIFF override (reason required) · approval email L1/L2 · bulk edits (confirm + undo)

DATA & SECURITY

100% client-side — bank data never leaves the Treasury laptop; public URL serves an empty shell; CSV outputs formula-escape hardened

COST TO RUN

≈ $0 — no server, no license; Python + a browser

GAPS TO PRODUCTION

single-user by design today → needs SSO/auth, server-side audit store, system-enforced maker-checker

STATUS & SCALE

Working today for one team — designed for regional treasury

Not a concept: it runs on real batches now, and the path to production and reuse is concrete.

TODAY — WORKING, DEMO-ABLE LIVE

Full lifecycle on real batches: 46 groups · 7 entities · 6 currencies · 3 banks · exception workflow (owner / age / status) · audit log of every action · QA-audited (WCAG AA, 14 findings fixed) · 2026 run calendar built in (35 run days, 17 blocked Wednesdays)

TO GO LIVE — SCOPED

UAT with Treasury users on 2–3 real cycles → user access control (SSO) → system-enforced maker-checker → production hosting + server-side audit store → data security review

SCALE PATH

New entity = configuration, not code · more banks via the same rule contract · reusable by any FA / Treasury team · Regional Treasury Control Tower

ROADMAP

Auto approval package	Zalo / Teams alerts	Holiday alert agent	Bank API integration	Cash-forecast link	Regional roll-up

Live now: tultc4.github.io/treasury-payment-dashboard